# Структурное тестирование многочлена Лагранжа

## Лабораторная работа 3 Вариант 3

Дата выполнения: 03.11.2026. Тема: структурное тестирование и тестирование белого ящика.

## Цель и результат работы

Изучить тестирование базового пути на фрагменте вычисления элементарного интерполяционного многочлена. Построен потоковый граф из семи вершин и восьми дуг. Его цикломатическая сложность равна 3. Подготовлены три независимых пути графа, восемь примеров на допустимых данных и отдельный структурный эксперимент.

Все 22 автоматические проверки ЛР3 прошли. Вместе с проверками калькулятора выполнено 65 тестов, ошибок и пропусков нет. Для допустимых примеров подтверждены все восемь дуг графа. Путь с нулём итераций отмечен как недостижимый в математической области входных данных.

## Математическая постановка

Даны n + 1 различных узлов $x_0$, …, $x_n$, индекс k и точка t. Требуется вычислить один элементарный многочлен $L_k(t)$. Значения функции в узлах не нужны: весь интерполяционный многочлен здесь не строится.

$$L_k(t) = \prod_{i = 0,\, i \neq k}^{n} \frac{t - x_i}{x_k - x_i}$$

Допустимые данные: $n \geq 0$, $0 \leq k \leq n$; узлы попарно различны. В программе используются конечные значения типа double. Пустой массив, неверный индекс, повторяющиеся узлы и NaN или бесконечность отклоняются до вычисления. При n = 0 произведение не содержит множителей и равно 1.

## Краткие теоретические сведения

При тестировании белого ящика тесты выбирают с учётом исходного кода. Покрытие операторов требует выполнить каждый оператор, покрытие решений - каждый исход условия, покрытие условий - оба значения простых условий. Покрытие решений и условий объединяет эти требования. Комбинаторное покрытие проверяет возможные сочетания значений условий.

Метод базового пути использует граф управления. Вершина представляет линейный участок кода или условие, дуга - возможный переход. Независимый путь добавляет дугу, отсутствующую в ранее выбранных путях. Цикломатическая сложность задаёт размер базиса графа; достижимость его путей проверяется по значениям переменных.

## Исходный алгоритм и нумерация

Сохранены цикл, проверка i == k, переход continue и порядок перемножения из варианта 3 методички. В листинге опущены ввод, проверка предусловий и регистрация маршрута.

```
L = 1.0;                    // вершина 1
i = 0;
while (i <= n) {             // вершина 2
    if (i == k) {            // вершина 3
        ++i;                 // вершина 4
        continue;
    }
    L *= (t - x[i]) / (x[k] - x[i]); // вершина 5
    ++i;                     // вершина 6
}
// Ответ L                   // вершина 7
```

| Вершина | Действие или условие |
|---|---|
| 1 | L = 1.0; i = 0 - начальные значения |
| 2 | i <= n - условие продолжения цикла |
| 3 | i == k - требуется пропустить k-й множитель |
| 4 | ++i; continue - перейти к следующей итерации |
| 5 | L *= (t - x[i]) / (x[k] - x[i]) |
| 6 | ++i после вычисления множителя |
| 7 | Завершение вычисления и возврат L |

## Границы структурного анализа

Метрика и покрытие относятся к вычислительному фрагменту EvaluateFragment в файле Lab3/LagrangeBasis.cs. Оболочка Calculate проверяет входные данные, а консольная программа отвечает за ввод и вывод; эти участки не включены в граф. Их ошибки проверены отдельными тестами.

Для наблюдения за маршрутом функция Check записывает номер проверяемой вершины и возвращает исходное значение условия. Записи Path не меняют вычислительный алгоритм. Вершина 2 регистрируется и при последней, ложной проверке цикла. Вершина 4 ведёт прямо к вершине 2, поэтому после continue умножение и вершина 6 пропускаются.

## Потоковый граф и цикломатическая сложность

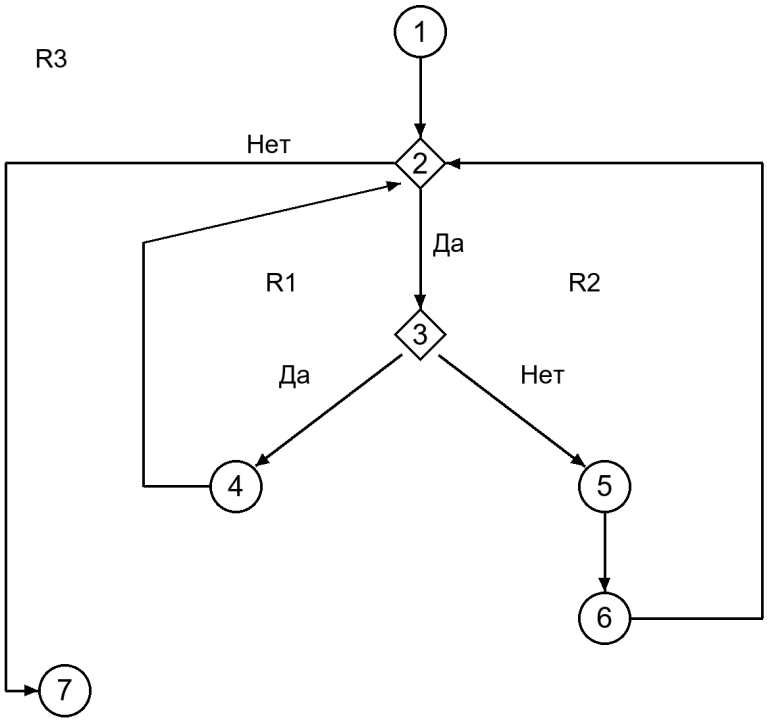



Рисунок 1. Потоковый граф вычислительного фрагмента. Ромбы 2 и 3 обозначают предикатные вершины. R1 и R2 - внутренние области, R3 - внешняя область.

Дуги: 1 → 2; 2 → 3; 2 → 7; 3 → 4; 3 → 5; 4 → 2; 5 → 6; 6 → 2. Граф связный, имеет один вход и один выход.

| Способ расчёта | Расчёт | Результат |
|---|---|---|
| По дугам и вершинам | V(G) = E - N + 2 = 8 - 7 + 2 | 3 |
| По предикатным вершинам | V(G) = P + 1 = 2 + 1 | 3 |
| По областям плоского графа | R1 + R2 + R3 | 3 |

Все три способа дают V(G) = 3. Число означает размер структурного базиса, а не три возможных полных запуска: число маршрутов с разным количеством итераций зависит от n.

## Базовое множество независимых путей

P1: 1 → 2 → 7. Цикл не выполняется. После i = 0 условие i <= n ложно только при n < 0. В допустимой области этот путь недостижим. Эксперимент S01 вызывает только исходный внутренний фрагмент с n = -1 и получает L = 1; это не корректная задача интерполяции.

P2: 1 → 2 → 3 → 4 → 2 → 7. Данные T01: x = [3], n = 0, k = 0, t = 100. Единственный множитель пропускается, L = 1. Новые дуги относительно P1: 2 → 3, 3 → 4 и 4 → 2.

P3: 1 → 2 → 3 → 5 → 6 → 2 → 3 → 4 → 2 → 7. Данные T02: x = [0; 2], n = 1, k = 1, t = 1. Сначала вычисляется множитель 1/2, затем пропускается индекс k. Новые дуги: 3 → 5, 5 → 6 и 6 → 2.

Каждый следующий путь добавляет новые дуги, поэтому пути независимы. Их число равно V(G). На допустимых данных невозможно воспроизвести P1, но T02 уже покрывает все дуги и оба исхода каждого из двух условий. Покрытие дуг не требует пропускать цикл при входе.

## Тестовые данные и фактические результаты

Здесь n равен числу узлов минус один. Результаты получены запуском реализации C#; сравнение выполняется с абсолютным допуском $10^{-12}$. Все восемь примеров прошли.

| Тест | Узлы x | k | t | Ожидалось | Получено |
|---|---|---|---|---|---|
| T01 | 3 | 0 | 100 | 1 | 1 |
| T02 | 0; 2 | 1 | 1 | 0,5 | 0,5 |
| T03 | 0; 2 | 0 | 1 | 0,5 | 0,5 |
| T04 | 0; 1; 2 | 1 | 1,5 | 0,75 | 0,75 |
| T05 | 0; 1; 2 | 1 | 1 | 1 | 1 |
| T06 | 0; 1; 2 | 1 | 0 | 0 | 0 |
| T07 | -2; 0; 2 | 1 | 3 | -1,25 | -1,25 |
| T08 | -1; 0; 2; 4 | 2 | 1 | 0,5 | 0,5 |

Отдельно S01: n = -1, x = [], k = 0, t = 0. Ожидалось 1; получено 1. Зарегистрирован путь 1 → 2 → 7. Публичный метод Calculate отклоняет пустой массив, что подтверждено тестом.

## Разбор вычисления и проверка покрытия

В T04 заданы x = [0; 1; 2], k = 1, t = 1,5. При i = 0 получаем множитель (1,5 - 0)/(1 - 0) = 1,5. При i = 1 выполняется continue. При i = 2 множитель равен (1,5 - 2)/(1 - 2) = 0,5. Итог: 1,5 × 0,5 = 0,75.

Фактический маршрут T04: 1 → 2 → 3 → 5 → 6 → 2 → 3 → 4 → 2 → 3 → 5 → 6 → 2 → 7.

T03 проверяет пропуск первого индекса, T05 - значение 1 в собственном узле, T06 - значение 0 в другом узле. T07 проверяет экстраполяцию и отрицательный результат, T08 - цикл с четырьмя узлами. Дополнительный тест проверяет тождество суммы базисных многочленов: сумма $L_k(t)$ по всем k равна 1.

Покрытие рассчитывается по фактически зарегистрированным соседним вершинам Path, а не по списку заранее ожидаемых переходов. В допустимых примерах обнаружено 8 из 8 дуг и 4 из 4 исходов условий. Это покрытие выбранной модели; оно не является измерением покрытия всей сборки и не доказывает отсутствие ошибок.

## Результат автоматического прогона

MSTest: 65 из 65 тестов прошли. Из них 22 проверки ЛР3: 8 примеров, 4 проверки маршрутов, 1 проверка покрытия, 1 проверка математического тождества и 8 проверок ошибок входа или диапазона. Ещё 43 теста относятся к калькулятору.

Проверены отклонение null и пустого массива, двух недопустимых индексов, повторяющихся узлов, NaN в точке t, бесконечного узла и результата вне диапазона double. Для обычных данных точность ограничена арифметикой double; близкие узлы и экстремальные значения могут увеличивать численную ошибку.

## Запуск из директории решения

```
dotnet run --project Lab3/SoftwareTesting.Lab3.csproj
# Ввести узлы через пробел, затем k и t

# Пример с аргументами: k=1, t=1, x=[0; 2]
dotnet run --project Lab3/SoftwareTesting.Lab3.csproj -- 1 1 0 2

# Все тесты решения
dotnet test SoftwareTesting.sln
```

В Visual Studio выбрать проект SoftwareTesting.Lab3 в обозревателе решения и назначить его запускаемым. Для вывода сохранённых примеров использовать аргумент --demo. Полные результаты находятся в Docs/Results/2026-11-03/lab3-cases.json и lab3.trx.

Дополнительно прошли семь проверок консольного запуска: аргументы, дробь с запятой, ввод по запросам, повторяющиеся узлы, неверный индекс, некорректное число и справка. Ввод, вывод и коды возврата сохранены в lab3-cli.json рядом с остальными результатами.

## Ответы на контрольные вопросы

### 1 Понятие тестирования

Тестирование - процесс проверки программного обеспечения, включающий подготовку и выполнение проверок, оценку результатов и выявление расхождений с ожидаемым поведением. Успешный прогон выбранных тестов не доказывает отсутствие всех дефектов.

### 2 Тест и процесс тестирования

Тест задаёт входные данные, предусловия, действия и ожидаемый результат. Процесс включает анализ объекта, выбор проверок, подготовку среды, выполнение, сравнение результатов, регистрацию дефектов и повторную проверку исправлений.

### 3 Структурное тестирование

Это тестирование, при котором сценарии выбирают по внутренней структуре программы. Для управления потоком используют граф, условия, ветви и циклы; полноту оценивают по выбранному критерию покрытия.

### 4 Суть тестирования белого ящика

Тестировщик знает исходный код и подбирает данные для выполнения конкретных операторов и переходов. Проверяются внутренние элементы и их взаимодействие, а результаты сопоставляются с ожидаемыми.

### 5 Особенности белого ящика

Проверяют независимые пути, оба исхода решений и допустимые режимы циклов. При составных условиях учитывают порядок вычисления и короткое замыкание. Достижимость пути устанавливают по данным и состоянию программы.

### 6 Достоинства и недостатки

Метод позволяет находить ошибки ветвления, пропущенные переходы и ошибки работы циклов, а также измерять покрытие кода. Он требует доступа к реализации; число маршрутов может быть большим, а отсутствующую в коде функциональность или ошибку требований полное покрытие само по себе не выявляет.

## Ответы на контрольные вопросы продолжение

### 7 Стратегии белого ящика

Покрытие операторов требует выполнить каждый оператор. Покрытие решений - получить оба исхода каждого решения. Покрытие условий - получить истину и ложь для простых условий. Покрытие решений и условий объединяет эти проверки; комбинаторное покрытие рассматривает все достижимые сочетания условий. Метод базового пути выбирает структурно независимые маршруты.

### 8 Тестирование базового пути

По графу управления вычисляют цикломатическую сложность, выделяют базис независимых путей, подбирают данные и выполняют проверки. Нереализуемые пути отмечают с объяснением ограничений; их нельзя объявлять проверенными на допустимых данных.

### 9 Особенности потокового графа

Вершины соответствуют линейным участкам кода и условиям, ориентированные дуги - переходам управления. У простого предиката два исхода. Составное условие разбивают на простые проверки с учётом короткого замыкания. При подсчёте областей учитывают внешнюю область.

### 10 Цикломатическая сложность

Это количественная характеристика структуры управления и размер независимого базиса графа. Для связного графа с одним входом и выходом $V(G) = E - N + 2$; при бинарных предикатах также $V(G) = P + 1$. Для плоского изображения результат равен числу областей с внешней.

### 11 Базовое множество

Базовое множество состоит из независимых путей графа. В выбранном порядке каждый следующий путь содержит ранее не использованную дугу. В данной работе P1, P2 и P3 образуют структурный базис; P1 нельзя реализовать в математической области $n \geq 0$.

### 12 Шаги метода базового пути

Пронумеровать участки исходного кода, построить граф, рассчитать сложность, выделить независимые пути и проверить их достижимость. Затем подобрать входные данные и ожидаемые результаты, выполнить тесты, сравнить фактическое поведение и оформить выводы.

### Вывод

Для вычисления $L_k(t)$ построен граф, тремя способами получена сложность 3 и выделен структурный базис. Фактические значения и маршруты совпали с ожидаемыми. Все дуги вычислительного графа покрыты допустимыми примерами; ограничение пути с нулём итераций явно отражено в результатах.

### Источник

Тестирование ПО.pdf. Методические указания. Лабораторная работа 3: с. 12-29; порядок выполнения: с. 26; вариант 3: с. 27; требования к отчёту и контрольные вопросы: с. 29.